

Amaya: Retrieval-Augmented Generation (RAG) based Document Querying System in an Air-Gapped Environment*

1st Nandini Gandhi

Terminal Ballistics Research Laboratory
DRDO

Chandigarh, India
AI Research Intern

2nd Samriti Gupta

Terminal Ballistics Research Laboratory
DRDO

Chandigarh, India
Sc 'E'

3rd Ashish Kumar

Terminal Ballistics Research Laboratory
DRDO

Chandigarh, India
TO 'B'

Abstract—This paper details the development of Amaya, a sophisticated, local-first Retrieval-Augmented Generation (RAG) platform designed for secure, air-gapped deployment on decentralized hardware. Engineered to address the critical challenges of data sovereignty and document structural integrity, the system implements a layout-aware ingestion pipeline utilizing the IBM Docling framework to accurately parse complex PDF geometries, including mathematical notation enriched via LaTeX and tabular data reconstructed through TableFormer V2 [1]. The architecture employs a multi-stage retrieval strategy, integrating Ollama-based dense embeddings with a local Cross-Encoder for neural re-ranking against a persistent ChromaDB vector store [2], [3]. Final response synthesis is facilitated by a localized Qwen 2.5 3B model, which generates evidence-based insights through a custom Streamlit executive dashboard, ensuring that every assertion is anchored to specific document provenance including section breadcrumbs and page numbers. By strictly enforcing offline operational protocols and manual dependency management, Amaya provides a robust and scalable solution for high-security environments where data privacy and analytical precision are paramount.

Index Terms—Retrieval-Augmented Generation, Air-Gapped Networks, Layout-Aware Parsing, Natural Language Processing, Data Sovereignty, Local LLM

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has transformed how organizations extract and synthesize information. However, standard LLMs are constrained by their static training data and suffer from hallucination when queried on proprietary or highly technical subjects. Retrieval-Augmented Generation (RAG) resolves this by dynamically grounding the model's responses in a searchable external knowledge base.

Despite the efficacy of RAG, enterprise and research environments face a critical bottleneck: data privacy and security. Standard RAG architectures rely heavily on cloud-based APIs for embedding generation, vector storage, and text inference. For organizations operating under strict compliance protocols, data sovereignty laws, or air-gapped security require-

ments, transmitting proprietary documents to external servers is strictly forbidden. This challenge is particularly acute at the Terminal Ballistics Research Laboratory (TBRL), a key DRDO unit specializing in high explosives, detonics, and shock wave dynamics, operating across a 5,000-acre range. Operations within such strategic armament evaluation centers require stringent confidentiality protocols and end-to-end local capabilities.

To address these limitations, Amaya was developed as a next-generation Operational Intelligence Platform. It is engineered to deliver high-fidelity, hallucination-free insights entirely on local hardware, ensuring absolute data containment and privacy.

Furthermore, conventional data extraction methodologies frequently overlook deep semantic associations, which diminishes retrieval precision and degrades the contextual relevance of synthesized insights. Standard document intelligence pipelines often struggle to maintain the structural hierarchy and logical organization of complex assets. When geometric information is compromised, the overall contextual integrity is lost, resulting in fragmented data retrieval and suboptimal response generation.

The primary objective of this project is to design, implement, and evaluate an advanced, layout-aware RAG pipeline that operates independently of the internet. The system execution is strictly localized, utilizing internal environment variables such as `HF_HUB_OFFLINE="1"` to disable outbound network requests. Ultimately, it provides a foundation for verifiable analytical synthesis by explicitly mapping generative outputs to precise document metadata, thereby serving as a viable document querying solution tailored for high-security enterprise applications in restricted zones.

II. SYSTEM ARCHITECTURE AND METHODOLOGY

The Amaya platform is engineered with a modern Python-centric technology stack, explicitly designed to function within high-security, air-gapped environments. Unlike traditional web applications that rely on cloud-based microservices, this archi-

*This work was carried out during a six-month industrial training at Terminal Ballistics Research Laboratory (TBRL), DRDO, Chandigarh.

texture utilizes a decentralized, local-first topology to guarantee absolute data containment.

A. Air-Gapped Environment Configuration

To comply with the stringent security protocols of a strategic research facility, the system deployment requires strict offline dependency management. The environment setup process initiates on an online machine, where a virtual environment specifically for Python 3.11 is created using Conda. Following the installation of required libraries via a `requirements.txt` file, the environment is packaged into a zip archive (`myenv_archive.zip`) utilizing the `conda-pack` utility.

Concurrently, essential pre-trained models—including Docling components, a cross-encoder for re-ranking, embedding models, and generation models—are manually downloaded and placed into a dedicated local cache directory (`C:\docling_models\models_cache`) to serve as localized execution fallbacks. The packed environment and the downloaded models are subsequently transferred and extracted on the offline host machine. Execution relies on activating the environment locally via the `.\Scripts\activate.bat` command from within the extracted directory.

Configuration paths are explicitly defined in the `config.py` file, requiring precise setup for `BASE_DIR` and `OLLAMA_BASE_URL` to prevent execution errors. Absolute data containment is further guaranteed by forcing offline execution protocols via environment variables, specifically setting `HF_HUB_OFFLINE="1"`, `TRANSFORMERS_OFFLINE="1"`, and `NO_PROXY="*"`. Finally, the frontend user interface is launched using the extracted Python executable to run the `Streamlit gui.py` script [5].

B. Frontend Orchestration and UI Design

The frontend interface is developed using Streamlit, featuring a responsive, multi-page corporate dashboard tailored for continuous operations [5]. Styled with a “Minimalist Luxe” aesthetic utilizing custom CSS, it is divided into distinct operational hubs. The *Executive Overview* serves as a telemetry dashboard for monitoring operational health and semantic index counts. The *Knowledge Engineering Studio* acts as an ingestion hub for processing documents into isolated Knowledge Domains (vaults). Lastly, the *Cognitive Research Lab* provides an interactive, multi-vault chat interface capable of cross-referencing domains.

To prevent heavy document parsing from freezing the interface, the frontend employs a custom `JobManager` utilizing Python’s `threading.Lock()` and background queues (`threading.Thread`). This allows real-time progress bars and asynchronous status updates to render dynamically without main-thread blocking.

C. Layout-Aware Ingestion Pipeline

Conventional document extraction techniques frequently fracture complex layouts and corrupt mathematical formulas.

To preserve document geometry, Amaya implements a layout-aware ingestion pipeline utilizing the IBM Docling framework alongside PyMuPDF (`fitz`) for initial metadata extraction [1].

The `IngestionEngine` module configures `PdfPipelineOptions` to execute structural parsing. Multi-column tables are reconstructed using `TableFormerV2` configured in “accurate” mode, exporting them into functional CSV matrices. Furthermore, images are isolated into standalone directories to prevent data corruption (`generate_picture_images = True`). Additionally, the pipeline employs `CodeFormulaV2` to extract and translate mathematical equations directly into valid LaTeX syntax, ensuring structural logic is preserved [1].

Extracted documents are subsequently segmented using either a `HybridChunker` (context-aware windowing) or a `HierarchicalChunker` (strict structural tagging), with a default maximum token capacity of 512. Each generated text chunk is appended with highly structured Pydantic metadata capturing the source name, page numbers, hierarchical breadcrumbs, and boolean flags for tables and formulas.

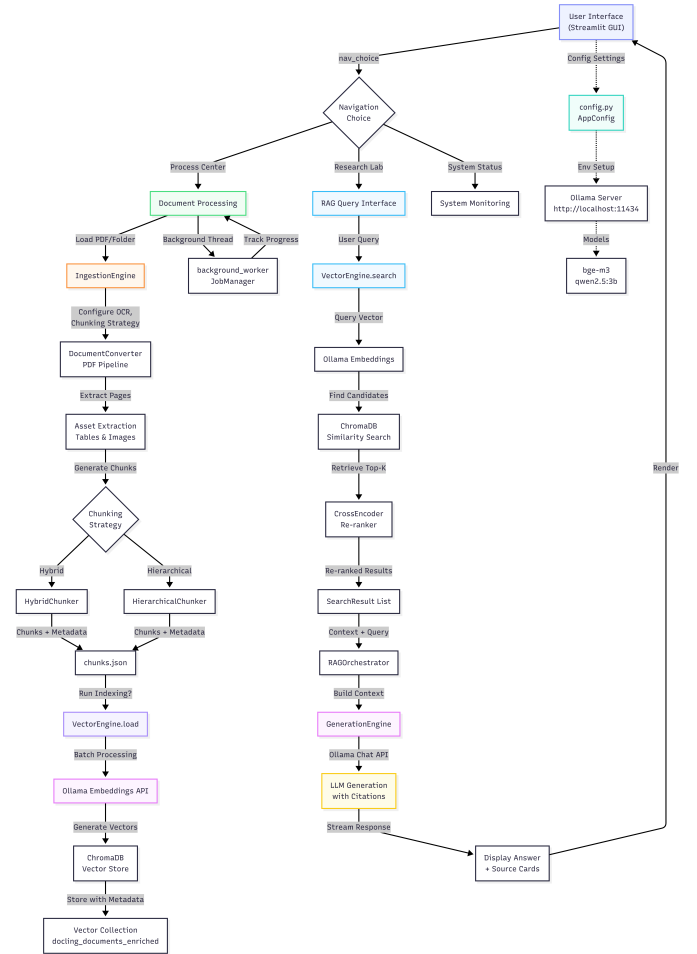


Fig. 1. High-level routing and logical data transformations of the Amaya system architecture.

D. Vector Indexing and Neural Re-ranking

Following the structural ingestion of documents, the `VectorEngine` manages the storage and retrieval of vectorized knowledge. Text segments are batched into arrays of 32 and transmitted to the local Ollama server, where the `bge-m3` model converts them into multi-dimensional dense arrays. These embeddings, along with their hierarchical structural metadata, are persistently stored on-disk using a local `ChromaDB` client configured for cosine similarity metrics [2].

To mitigate the limitations of basic cosine similarity—which often returns candidates sharing overlapping keywords but lacking true semantic relevance—Amaya implements a Hybrid-Compute Retrieval Topology. Upon receiving a user query, `ChromaDB` executes a rapid initial search to retrieve a broad pool of candidates. These candidates are subsequently passed through a local neural re-ranker, specifically a Cross-Encoder (`ms-macro-MiniLM-L6-v2`) [3]. This model evaluates the logical relationship between the prompt and the document context, dynamically re-scoring and filtering the chunks for maximum precision before generation.

E. Generation Engine and Provenance Tracking

To eliminate hallucinations common in large language models, the system strictly enforces data traceability. The `RAGOrchestrator` builds a rich context block from the re-ranked candidates, purposefully injecting the Pydantic-validated metadata headers directly into the prompt.

The final payload is routed to a localized language model (`Qwen2.5-3B-Instruct`) [4]. The `GenerationEngine` utilizes a strict system prompt that forces the LLM to stream its synthesized answer with mandatory bracketed citations (e.g., [Source X]) for every factual assertion. These citations are parsed by the Streamlit user interface into expandable “Evidence Registry & Provenance” cards, allowing operators to instantly verify generated claims against the raw PDF excerpts.

III. TESTING AND VALIDATION

To guarantee the reliability, precision, and security of the Amaya platform within a mission-critical, air-gapped environment, a rigorous, multi-tiered testing methodology was executed.

Testing was conducted across four primary vectors:

- **Unit Testing:** Focused on validating individual components independently, such as verifying Pydantic schema constraints and environmental overrides.
- **Integration Testing:** Evaluated end-to-end data pipelines, specifically the ingestion-to-vectorization loop and retrieval-to-generation loop managed by the `RAGOrchestrator`.
- **User Interface (UI) Testing:** Verified the stability of the Streamlit frontend under heavy file loads, ensuring that multi-threaded background workers prevented main thread freezing.

- **Security and Air-Gap Testing:** Validated that zero external telemetry or network leakages occurred by monitoring system sockets while processing sensitive inputs. Database parameter safety was also verified to ensure queries executed by `ChromaDB` were strictly object-relation driven, blocking arbitrary code execution vectors.

Specifically, Outbound Network Isolation Testing utilizing socket sniffers and packet analysis utilities confirmed zero outgoing external web requests. All model queries were strictly and successfully routed to internal network boundaries (e.g., `http://localhost:11434`).

IV. RESULTS AND PERFORMANCE EVALUATION

The deployment of the Amaya platform successfully demonstrated that enterprise-grade, high-fidelity AI intelligence can be generated in a strictly offline environment. The implementation of the IBM Docling pipeline effectively resolved semantic fragmentation by flawlessly reconstructing multi-column tables and LaTeX mathematical formulas. Users highlighted the high usability of the Evidence Registry cards, which allowed immediate cross-verification of AI-generated text against raw PDF excerpts.

A. System Latency and Resource Utilization

Performance evaluation was conducted across local hardware limits to measure the computational cost of the multi-stage pipeline. The addition of the lightweight Cross-Encoder re-ranking phase provided an optimal balance, delivering highly precise semantic filtering without overloading host memory.

TABLE I
SUBSYSTEM PROCESSING LATENCY PROFILE

Pipeline Operation	Tested Scope	Mean Response Latency
Structure Extraction	Single 15-Page PDF	≈ 12.4 seconds
Vector Generation	50 Chunks Batched	≈ 3.2 seconds
Vector Query	Global Search	< 15 milliseconds
Neural Re-ranking	20 Candidates	≈ 420 milliseconds
Inference Stream	512 In / 150 Out Tokens	≈ 8.5 seconds

^aBenchmarked on base 16GB RAM hardware configuration.

As detailed in Table I, the local vector query operates in under 15 milliseconds, and the critical neural re-ranking phase adds only ≈ 420 milliseconds of latency for 20 candidates. While layout-aware vision parsing and language model generation cause momentary spikes in CPU utilization, the system remains highly responsive due to the asynchronous `JobManager` thread locks implemented in the user interface.

V. FUTURE SCOPE

While the platform met its core objectives, the modular architecture provides a strong foundation for future scalability. Proposed enhancements include:

- **Hybrid Search Implementation:** Integrating BM25 (keyword-based) indexing with dense vectors using reciprocal rank fusion (RRF) to retrieve highly specific technical identifiers alongside broad semantic concepts.

- **Role-Based Access Control (RBAC):** Developing an authorization matrix to separate user personas (Administrator, Contributor, Researcher) and restrict read-only query access across specific knowledge vaults.
- **Multi-Modal AI Integration:** Upgrading the local inference engine to support Large Vision-Language Models (VLMs) like LLaVA or Qwen-VL, allowing users to query extracted diagrams and technical schematics.
- **Agentic Self-Correction Loops:** Implementing a secondary, hidden LLM verification loop to force the model to self-audit its proposed answer against retrieved context chunks before displaying it.
- **Hardware Acceleration:** Optimizing execution frameworks utilizing TensorRT or vLLM to seamlessly detect and utilize multi-GPU clusters for larger organizational deployments.

VI. CONCLUSION

The Amaya Digitalized Retrieval-Augmented Generation project establishes a robust, zero-trust software architecture tailored for strategic research facilities. By abandoning linear text-splitters in favor of a layout-aware ingestion pipeline, the system successfully maintains the geometric and contextual integrity of complex technical documentation.

Furthermore, the integration of a hybrid-compute retrieval topology with Cross-Encoder re-ranking drastically elevates search precision, while strict prompt engineering ensures all generated intelligence is hallucination-free and verifiable. Ultimately, Amaya proves that advanced document querying systems can operate with high efficacy within strict air-gapped constraints, ensuring total data sovereignty and privacy.

REFERENCES

REFERENCES

- [1] IBM Docling Core Framework, "Layout-Aware Document Processing & Structural Parsing Reference Guide," URL: <https://github.com/DS4SD/docling>.
- [2] ChromaDB, "AI-Native Persistent Embedding Database Management Guide," URL: <https://docs.trychroma.com/>.
- [3] Sentence Transformers, "Cross-Encoder Models for High-Precision Neural Text Re-ranking," URL: <https://www.sbert.net/>.
- [4] Alibaba Cloud Qwen Research Team, "Qwen2.5-3B-Instruct Lightweight Foundation Model Architecture Profile," URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- [5] Streamlit Application Web Framework, "Rapid Machine Learning Dashboard Design and Execution Methodology," URL: <https://docs.streamlit.io/>.